

Comunicação Síncrona e Assíncrona em MPI: Análise de Desempenho na Multiplicação de Matrizes

Rayan Raddatz de Matos, Enzo Lisboa Peixoto, Eduardo Magnus Lazuta

Introdução

Este trabalho aborda três formas de comunicação de dados em aplicações desenvolvidas com MPI, comparando seu desempenho. Os modelos de comunicação analisados são os seguintes:

1. **Comunicação coletiva:** utiliza operações coletivas do MPI, empregando `MPI_Scatter` para distribuir as linhas da matriz (A), `MPI_Bcast` para difundir a matriz (B) e `MPI_Gather` para reunir a matriz resultante. Nessa abordagem, a biblioteca MPI decide internamente a estratégia mais eficiente para realizar a comunicação entre os processos.
2. **Comunicação ponto a ponto bloqueante (síncrona):** o processo mestre envia cada parcela dos dados utilizando `MPI_Send`, enquanto os processos trabalhadores as recebem por meio de `MPI_Recv`. Cada operação retorna apenas após a conclusão da transferência correspondente. A matriz (B) continua sendo distribuída com `MPI_Bcast`.
3. **Comunicação ponto a ponto não bloqueante (assíncrona):** utiliza `MPI_Isend`, `MPI_Irecv` e `MPI_Ibcast`, que apenas iniciam as operações de comunicação e retornam imediatamente. A sincronização ocorre posteriormente por meio de `MPI_Wait`, garantindo a conclusão das transferências antes da utilização dos dados.

O problema escolhido para a comparação foi a multiplicação de duas matrizes de dimensão $(N \times N)$. Nessa aplicação, o processo de rank 0 (mestre) divide as linhas da matriz (A) entre todos os processos, distribui a matriz (B) integralmente para cada um deles, cada processo calcula sua respectiva parcela da matriz resultante e, ao final, devolve o resultado ao processo mestre.

Metodologia

Os experimentos rodaram na partição **hype** do cluster PCAD (nós com 2x Xeon E5-2650 v3, 20 núcleos físicos por nó), com MPI sobre TCP e sem fixação de processos a núcleos (`--bind-to none`). Todas as combinações dos fatores abaixo foram executadas 2 vezes (324 execuções no total):

Fator	Níveis
Tipo comunicação	coletiva, p2p bloqueante, p2p não bloqueante
Nós	1, 2, 3
Processos por nó	1, 2, 4, 8, 16, 32
Tamanho (NxN)	500, 1000, 1500

Cada execução foi rastreada com a biblioteca `akypuera`: toda chamada MPI de todo processo fica registrada com início, fim e duração. As análises foram feitas a partir desses rastros. As métricas usadas são:

- **Tempo total:** medido pela própria aplicação com `MPI_Wtime` no processo 0 (do início da distribuição até o fim da coleta).
- **Tempo por função MPI:** para uma execução, somamos quanto cada processo passou dentro de cada função (por exemplo, todo o tempo de `MPI_Recv` do processo 3) e tiramos a média entre os processos. Isso responde: "em média, quanto um processo gasta nessa função?"

- Como cada configuração rodou 2 vezes, reportamos a mediana das duas e usamos o desvio padrão como indicação de variabilidade.

MPI_Finalize é excluído do tempo por função: ele só marca o fim do programa, e sua duração é apenas espera pelos processos mais lentos.

Linha do tempo das execuções (Gantt)

A forma mais direta de ver a diferença entre as versões é desenhar a linha do tempo de uma execução: cada linha horizontal é um processo e o tempo corre para a direita. Trechos coloridos são chamadas MPI (uma cor por função); trechos em branco são computação (o processo está multiplicando matrizes, sem comunicar). O cinza final é o MPI_Finalize (o processo já acabou e está esperando os demais).

Usamos a maior configuração do experimento – matriz 1500x1500, 96 processos em 3 nós (32 por nó) – porque é onde a comunicação mais aparece. Os processos 0 a 31 estão no mesmo nó que o mestre; os demais estão em nós remotos, acessados pela rede.

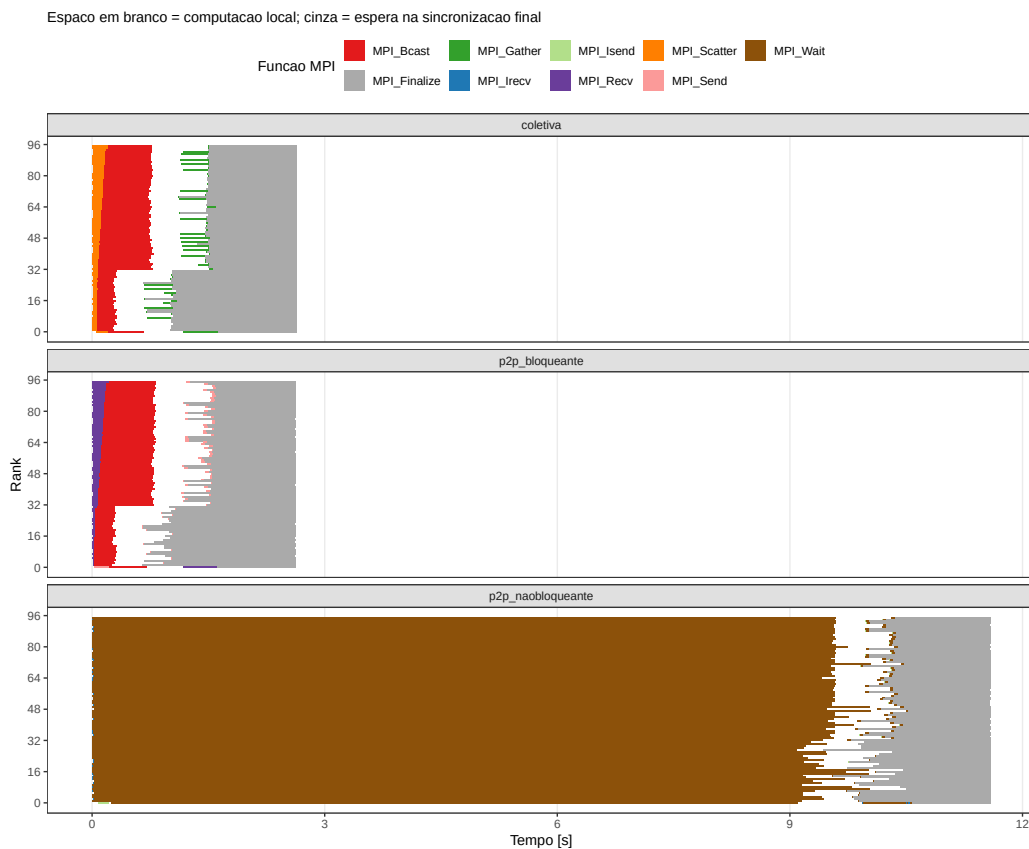


Figura 1: Uma execução de cada versão (size=1500, 96 processos, 3 nós). Cada linha horizontal é um processo.

Nas versões coletiva e bloqueante a comunicação ocupa só o começo e o fim, e tudo termina em cerca de 2.7s. Na versão não bloqueante quase todo o gráfico é marrom (MPI_Wait): os processos passam a maior parte dos ~10s de execução esperando dados chegarem.

Para ver as diferenças de semântica entre as versões, ampliamos os primeiros 0.6s (fase de distribuição dos dados):

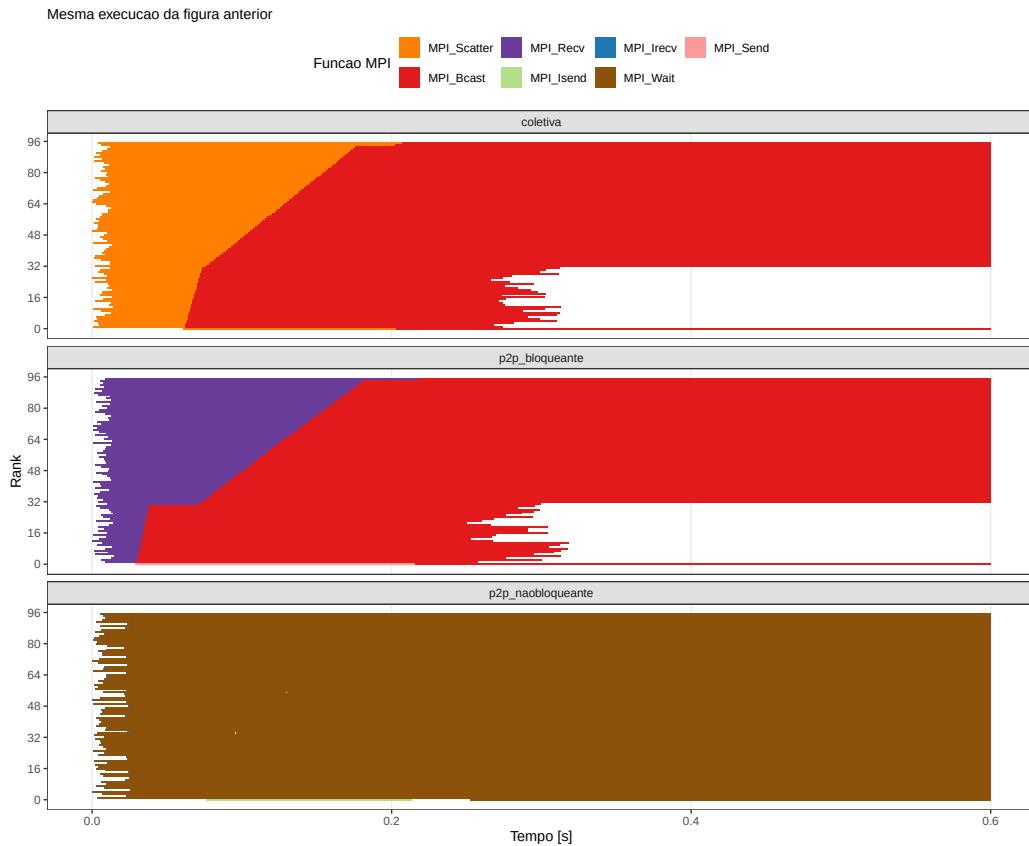


Figura 2: Zoom nos primeiros 0.6s da mesma execucao.

- **Coletiva:** todos entram juntos no **Scatter** (laranja) e depois no **Bcast** (vermelho). Note a fronteira de nó: os processos 0-31 (mesmo nó do mestre) saem do **Bcast** em ~ 0.3 s e começam a computar, enquanto que os processos remotos só saem em ~ 0.7 s, porque receber B pela rede demora mais.
- **Bloqueante:** os trabalhadores ficam parados no **Recv** (roxo) esperando sua vez, enquanto o mestre envia os 95 pedaços um por um, por isso a borda roxa é diagonal: quanto maior o rank, mais tarde o dado chega. Depois, o mesmo **Bcast** da coletiva.
- **Não bloqueante:** os **Irecv** retornam instantaneamente e a espera real acontece no **Wait** (marrom). É de se notar que o rastreador não registra o **MPI_Ibcast** como um estado separado, então a espera pela matriz B também está dentro desse **Wait**.

Tempo por operação MPI

O gráfico abaixo decompõe o tempo total por função MPI para size=1500, empilhando a contribuição de cada função. Permite ver de imediato qual função domina o custo em cada versão e configuração.

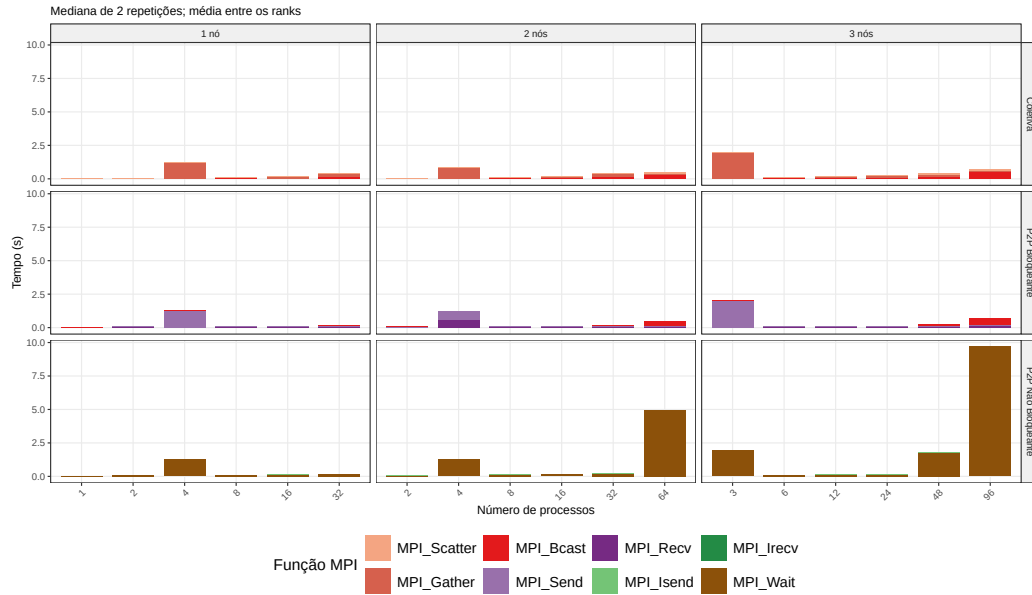


Figura 3: Tempo médio por processo em cada função MPI (size=1500, mediana de 2 repetições). Barras empilhadas por função.

O gráfico seguinte isola a comparação central do experimento: o **MPI_Wait** da versão não bloqueante (que engloba a espera real pela comunicação) versus o **MPI_Bcast** das versões coletiva e bloqueante (que fazem a mesma difusão de B, mas com algoritmo em árvore).

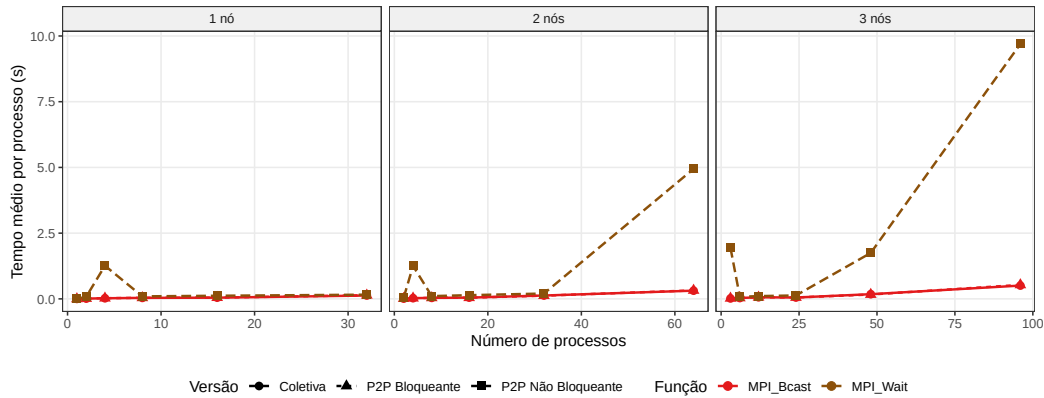


Figura 4: **MPI_Wait** (não bloqueante) vs **MPI_Bcast** (coletiva e bloqueante) para size=1500. O colapso do Wait em múltiplos nós é a causa da lentidão da versão assíncrona.

A tabela abaixo mostra o tempo médio por processo em cada função (mediana das 2 execuções), para size=1500 em 3 nós, nas duas maiores quantidades de processos:

Função	Versão	48 proc	96 proc
MPI_Wait	não bloqueante	1.75 s	9.70 s
MPI_Bcast	coletiva	0.18 s	0.51 s

Função	Versão	48 proc	96 proc
MPI_Bcast	bloqueante	0.17 s	0.53 s
MPI_Gather	coletiva	0.11 s	0.07 s
MPI_Scatter	coletiva	0.08 s	0.10 s
MPI_Recv	bloqueante	0.06 s	0.11 s
MPI_Send	bloqueante	0.03 s	0.03 s
MPI_Isend	não bloqueante	0.002 s	0.002 s
MPI_Irecv	não bloqueante	0.0005 s	0.0005 s

Três coisas a notar:

1. Quase todas as operações custam décimos de segundo ou menos, mesmo com 96 processos. A exceção é o **MPI_Wait** da versão não bloqueante: 1.75s com 48 processos e 9.7s com 96 – dobrar os processos multiplicou o custo por 5.5.
2. **Isend** e **Irecv** parecem "de graça" (milésimos de segundo), mas é ilusão: eles só iniciam a operação. O tempo que a comunicação realmente leva aparece depois, no **Wait**.
3. **MPI_Bcast** custa praticamente o mesmo na coletiva e na bloqueante: as duas chamam exatamente a mesma função da biblioteca.

Sobre como esses custos crescem: comparando execuções com o mesmo número de processos em 1, 2 ou 3 nós, os tempos de **Scatter**, **Bcast** e **Recv** quase não mudam – até ~32 processos, o que importa é quantos processos participam e quanto dado trafega, não quantos nós são usados. É acima de 32 processos (só possível com 2 ou 3 nós) que os custos coletivos crescem mais rápido e o **Wait** dispara.

Tabela completa: todas as combinações

A tabela abaixo traz o tempo médio por processo em cada função (em segundos, mediana de 2 execuções) para **todas** as combinações de tamanho, nós e processos. "Bcast (col)" e "Bcast (blq)" são o mesmo **MPI_Bcast** medido na versão coletiva e na bloqueante; "-" indica função não chamada (com 1 processo não há trocas ponto a ponto); "0.000" significa menos de 1 milissegundo.

size	nos	nproc	Scatter	Gather	Bcast (col)	Send	Recv	Bcast (blq)	Isend	Irecv	Wait
500	1	1	0.002	0.001	0.000	-	-	0.000	-	-	0.000
500	1	2	0.005	0.002	0.001	0.001	0.004	0.002	0.001	0.000	0.007
500	1	4	0.007	0.010	0.005	0.001	0.007	0.005	0.001	0.000	0.012
500	1	8	0.005	0.002	0.004	0.000	0.010	0.005	0.000	0.000	0.011
500	1	16	0.014	0.014	0.008	0.000	0.008	0.009	0.000	0.000	0.015
500	1	32	0.014	0.008	0.050	0.001	0.012	0.035	0.001	0.000	0.043
500	2	2	0.005	0.004	0.001	0.001	0.005	0.002	0.001	0.000	0.006
500	2	4	0.007	0.004	0.004	0.001	0.008	0.005	0.001	0.000	0.009
500	2	8	0.008	0.007	0.007	0.001	0.011	0.004	0.000	0.000	0.013
500	2	16	0.011	0.011	0.007	0.001	0.015	0.008	0.000	0.000	0.022
500	2	32	0.018	0.009	0.029	0.001	0.031	0.015	0.000	0.000	0.022
500	2	64	0.057	0.005	0.072	0.001	0.035	0.045	0.001	0.001	0.488
500	3	3	0.006	0.004	0.002	0.001	0.007	0.003	0.001	0.000	0.008
500	3	6	0.008	0.026	0.006	0.001	0.010	0.007	0.000	0.000	0.011
500	3	12	0.010	0.021	0.007	0.000	0.010	0.006	0.000	0.000	0.016
500	3	24	0.007	0.006	0.011	0.001	0.013	0.012	0.000	0.000	0.021
500	3	48	0.038	0.004	0.057	0.002	0.039	0.057	0.002	0.000	0.251
500	3	96	0.044	0.004	0.125	0.001	0.033	0.097	0.001	0.001	1.080
1000	1	1	0.005	0.003	0.000	-	-	0.000	-	-	0.000
1000	1	2	0.015	0.008	0.005	0.003	0.023	0.008	0.002	0.000	0.016
1000	1	4	0.019	0.081	0.012	0.002	0.025	0.016	0.002	0.000	0.088
1000	1	8	0.024	0.077	0.018	0.001	0.021	0.018	0.001	0.000	0.044

size	nos	nproc	Scatter	Gather	Bcast (col)	Send	Recv	Bcast (blq)	Isend	Irecv	Wait
1000	1	16	0.019	0.072	0.022	0.001	0.037	0.023	0.001	0.000	0.056
1000	1	32	0.033	0.059	0.079	0.002	0.033	0.070	0.001	0.000	0.097
1000	2	2	0.015	0.011	0.005	0.002	0.014	0.007	0.002	0.000	0.016
1000	2	4	0.020	0.124	0.013	0.002	0.016	0.014	0.002	0.000	0.038
1000	2	8	0.019	0.081	0.019	0.001	0.034	0.019	0.001	0.000	0.031
1000	2	16	0.025	0.036	0.024	0.001	0.046	0.026	0.001	0.000	0.088
1000	2	32	0.031	0.047	0.105	0.001	0.039	0.036	0.001	0.000	0.069
1000	2	64	0.046	0.023	0.146	0.013	0.077	0.167	0.002	0.001	2.204
1000	3	3	0.018	0.011	0.008	0.002	0.016	0.010	0.002	0.000	0.022
1000	3	6	0.018	0.064	0.016	0.002	0.025	0.021	0.002	0.000	0.046
1000	3	12	0.023	0.049	0.032	0.002	0.031	0.037	0.001	0.000	0.051
1000	3	24	0.027	0.049	0.028	0.001	0.033	0.026	0.001	0.000	0.060
1000	3	48	0.045	0.029	0.092	0.016	0.064	0.119	0.001	0.000	0.793
1000	3	96	0.065	0.020	0.248	0.001	0.072	0.271	0.002	0.001	4.361
1500	1	1	0.010	0.006	0.000	-	-	0.000	-	-	0.000
1500	1	2	0.030	0.034	0.010	0.052	0.023	0.014	0.003	0.000	0.084
1500	1	4	0.051	1.167	0.023	1.196	0.043	0.026	0.003	0.000	1.261
1500	1	8	0.035	0.057	0.038	0.002	0.085	0.041	0.002	0.000	0.096
1500	1	16	0.050	0.097	0.046	0.002	0.082	0.048	0.002	0.000	0.119
1500	1	32	0.052	0.216	0.128	0.002	0.079	0.138	0.002	0.000	0.160
1500	2	2	0.030	0.019	0.010	0.024	0.022	0.014	0.003	0.000	0.047
1500	2	4	0.038	0.813	0.027	0.605	0.620	0.026	0.003	0.000	1.253
1500	2	8	0.034	0.046	0.039	0.002	0.066	0.040	0.002	0.000	0.105
1500	2	16	0.052	0.133	0.048	0.001	0.063	0.044	0.002	0.000	0.141
1500	2	32	0.066	0.202	0.127	0.002	0.059	0.122	0.003	0.000	0.203
1500	2	64	0.106	0.100	0.312	0.021	0.107	0.323	0.002	0.001	4.945
1500	3	3	0.036	1.936	0.017	1.948	0.031	0.022	0.003	0.000	1.939
1500	3	6	0.038	0.066	0.033	0.003	0.061	0.035	0.003	0.000	0.079
1500	3	12	0.051	0.103	0.061	0.002	0.053	0.059	0.002	0.000	0.105
1500	3	24	0.052	0.181	0.052	0.001	0.088	0.053	0.001	0.000	0.131
1500	3	48	0.076	0.113	0.178	0.025	0.060	0.166	0.002	0.000	1.748
1500	3	96	0.103	0.073	0.506	0.025	0.109	0.533	0.002	0.001	9.696

O que a tabela mostra, coluna a coluna:

- Todas as funções bloqueantes ficam baratas em todas as combinações*: **Scatter**, **Gather**, **Bcast**, **Send** e **Recv** raramente passam de 0.1-0.2s, mesmo com 96 processos e matriz 1500. O custo cresce suavemente com o tamanho (mais bytes) e com o número de processos (mais participantes), mas nunca explode.
- A coluna **Wait** é a única que possui um comportamento explosivo, e de forma consistente nas três dimensões: cresce com o tamanho (com 64 processos: 0.49s em size=500, 2.20s em 1000, 4.95s em 1500), cresce com os processos (em size=1500: 0.20s com 32, 1.75s com 48, 4.95s com 64, 9.70s com 96) e só explode quando há mais de um nó – em 1 nó o **Wait** fica sempre abaixo de 0.16s. São exatamente as três assinaturas do problema do **Ibcast** descrito na seção de investigação: mais bytes, mais destinos e rede no caminho.
- Os valores altos isolados de **Gather**=\/=**Send** em size=1500 com 3-4 processos (~1.2-1.9s) não são custo de comunicação: são o mestre esperando trabalhadores que computam mais devagar (o desbalanceamento de memória citado na próxima seção) dentro da chamada de coleta.

Tempo total de execução

Média das 2 execuções, size=1500 (os tamanhos menores mostram as mesmas tendências).

O gráfico abaixo mostra o tempo total para todas as combinações de tamanho de matriz, número de nós e número de processos. Cada painel corresponde a um tamanho de matriz (linhas) e uma configuração de nós (colunas); o eixo x está em escala logarítmica.

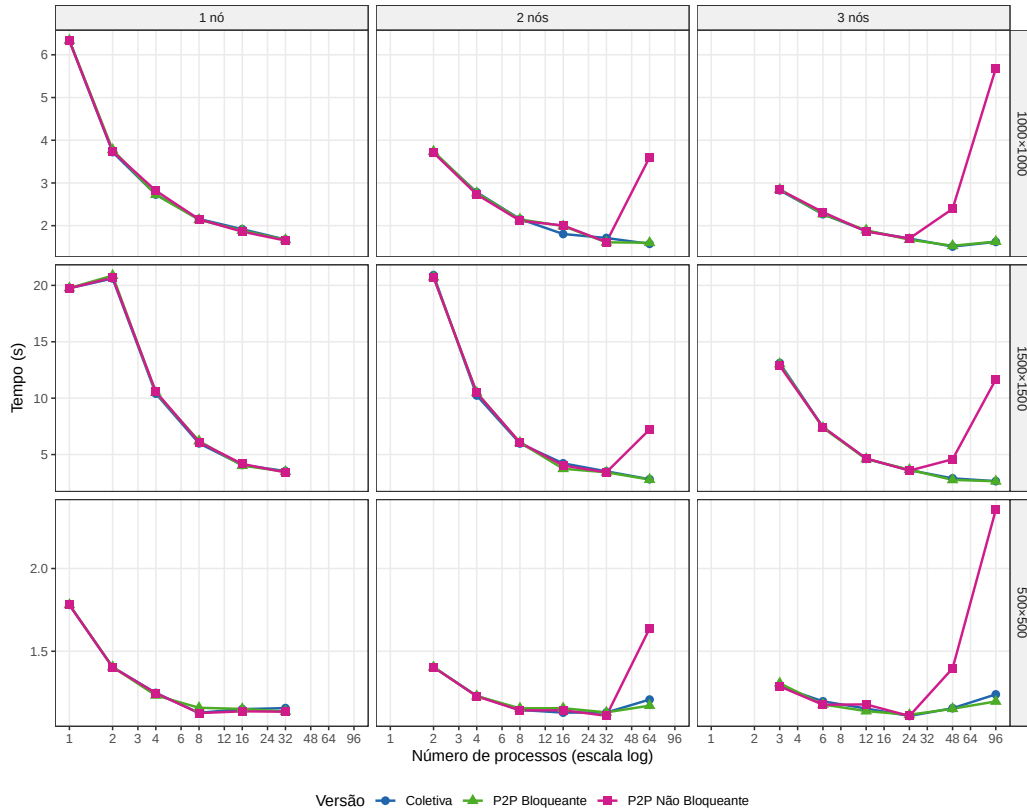


Figura 5: Tempo total de execução (média de 2 repetições) para todas as combinações experimentais. Eixo x em escala log.

O gráfico de escalabilidade abaixo fixa o número de processos por nó e varia o número de nós, isolando o efeito da comunicação inter-nó. As barras de erro indicam o desvio padrão entre as 2 repetições.

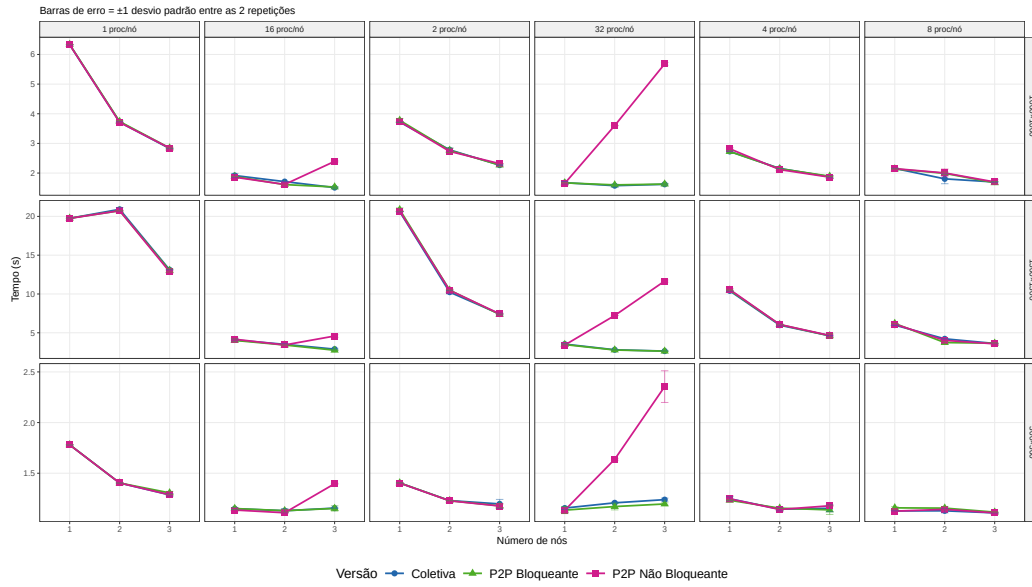


Figura 6: Escalabilidade por número de nós com número de processos por nó fixo. Barras de erro = ± 1 desvio padrão.

Nós	nproc	coletiva	p2p bloq.	p2p não bloq.
1	1	18.71 s	18.72 s	18.70 s
1	2	19.57 s	19.83 s	19.64 s
1	8	4.95 s	5.15 s	5.06 s
1	32	2.48 s	2.41 s	2.36 s
2	32	2.44 s	2.37 s	2.37 s
2	64	1.74 s	1.72 s	6.16 s
3	48	1.83 s	1.71 s	3.53 s
3	96	1.57 s	1.58 s	10.60 s

O speedup de cada versão em relação à execução sequencial (1 processo, 1 nó) é mostrado abaixo em escala $\log \times \log$. A linha tracejada cinza representa o speedup ideal (linear). Valores abaixo dela indicam ganho sub-linear; valores que caem abruptamente indicam overhead de comunicação superando o ganho de paralelismo.

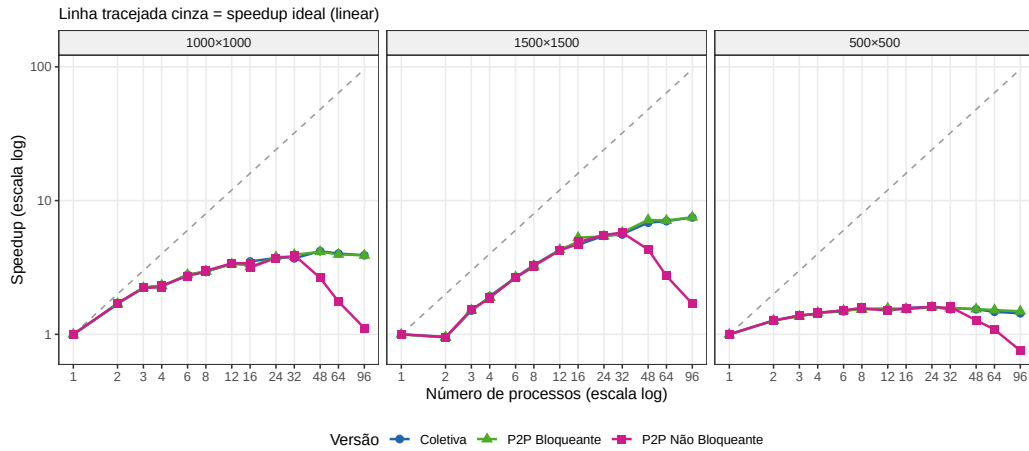


Figura 7: Speedup em relação à execução com 1 processo. Escala $\log \times \log$. Linha tracejada = speedup ideal.

Lendo a tabela de cima para baixo:

- Em um nó, as três versões empatam em todas as linhas: as diferenças ficam dentro da variabilidade entre repetições. Faz sentido: a multiplicação em si (o trecho branco do Gantt) leva segundos, enquanto toda a comunicação leva décimos de segundo. Quando a computação domina, o padrão de comunicação não importa.
- Uma curiosidade: com 2 processos o tempo *não cai* em relação a 1 (18.7s -> 19.6s). Isso só acontece no tamanho 1500 – nos tamanhos 500 e 1000 o tempo cai perfeitamente pela metade. Como o efeito é idêntico nas três versões, ele vem da máquina (disputa por memória/cache entre processos no mesmo nó, já que cada processo carrega uma cópia de B com 18MB) e não interfere na comparação entre os padrões.
- Com 2 e 3 nós, coletiva e bloqueante continuam melhorando (o melhor tempo do experimento é 1.57s, com 96 processos), mas **a versão não bloqueante piora muito**: 6.16s com 64 processos e 10.60s com 96 – quase 7x mais lenta que as outras duas.

Investigando o colapso da versão não bloqueante

Por que a versão assíncrona fica 7x mais lenta justamente na maior configuração? O rastro permite responder passo a passo.

Cada trabalhador chama `MPI_Wait` exatamente 3 vezes, nesta ordem: (1) esperar as linhas de A chegarem, (2) esperar a matriz B do `Ibcast`, (3) esperar o envio do resultado. Olhando um trabalhador da execução com 96 processos, as durações dos 3 `Wait` são:

Espera	Duração
(1) linhas de A (<code>Irecv</code>)	0.07 s
(2) matriz B (<code>Ibcast</code>)	9.1 s
(3) resultado (<code>Isend</code>)	~0 s

Ou seja: praticamente todo o tempo de `Wait` é espera pela matriz B. A mesma difusão de B, quando feita com o `MPI_Bcast` bloqueante, custa 0.5s (tabela da seção anterior). A diferença é a implementação: o `Bcast` bloqueante usa um algoritmo em árvore (o dado se propaga em paralelo, em $O(\log P)$ etapas), enquanto o `Ibcast` não bloqueante da biblioteca usa um esquema muito menos eficiente. Uma conta simples sustenta essa leitura: se o mestre enviasse B (18MB) individualmente para cada um dos 64 processos remotos por uma rede de 1Gb/s, levaria $18\text{MB} \times 64 / 125\text{MB/s} \sim 9.8\text{s}$ – exatamente a ordem do que medimos. Em 1 nó não há rede envolvida (memória compartilhada), por isso o problema só aparece em execuções multi-nó.

A lição não é "assíncrono é lento", e sim dupla: (1) trocar chamadas bloqueantes por não bloqueantes sem computar nada entre o início e o `Wait` apenas move a espera de lugar – o algoritmo aqui não sobrepõe computação e comunicação; e (2) as versões não bloqueantes das coletivas podem ter implementação bem pior que as bloqueantes, e isso só aparece quando a comunicação cruza a rede.

Conclusão

- Enquanto a computação domina (execuções em 1 nó), o padrão de comunicação é indiferente – e a coletiva vence pela simplicidade do código.
- Em múltiplos nós, coletiva e bloqueante escalam bem; a não bloqueante colapsa por causa do `MPI_Ibcast`, cuja implementação não usa a árvore otimizada do `Bcast` bloqueante.
- Comunicação assíncrona só compensa se o algoritmo computa enquanto espera. Sem essa sobreposição, ela no melhor caso empata e no pior caso perde por detalhes de implementação.

Para este algoritmo, a versão com operações coletivas (bloqueantes) é a preferível: desempenho igual ou melhor em todos os cenários, código mais simples e escalabilidade robusta.

Referências

Os experimentos deste trabalho utilizaram os recursos da infraestrutura PCAD, <http://pcad.inf.ufrgs.br>, no INF/UFRGS.